SQL TUNING

WHAT

PROCESS OF TUNING SQL STATEMENTS THAT ACCESS THE DATABASE

SQL TUNING

GOALS

TO FORMULATE STATEMENTS

TO ACCESS EFFECTIVELY THE DATABASE

TO UTILIZE OPTIMALLY SYSTEM RESOURCES AND INDEXES

SQL TUNING

EFFICIENT SQL STATEMENT

USING WHERE CLAUSE
FUNCTIONS
PROPER JOINS
USING THE CASE STATEMENT
BEST JOIN METHOD
USING IDENTICAL STATEMENTS
USING INLINE VIEWS
AVOID TABLE SCANS
USING PARTITIONED TABLES

Use a WHERE Clause to Filter Rows

BAD (retrieves all rows from the emp table)

SQL> select * from emp;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17 DEC 80	800		20
7499	ALLEN	SALESMAN	7698	20 FEB 81	1600	300	30
7521	WARD	SALESMAN	7698	22 FEB 81	1250	500	30
7566	JONES	MANAGER	7839	02 APR 81	2975		20
7654	MARTIN	SALESMAN	7698	28 SEP 81	1250	1400	30
7698	BLAKE	MANAGER	7839	01 MAY 81	2850		30
7782	CLARK	MANAGER	7839	09 JUN 81	2450		10
7788	SCOTT	ANALYST	7566	19 APR 87	3000		20
7839	KING	PRESIDENT		17 NOV 81	5000		10
7844	TURNER	SALESMAN	7698	08 SEP 81	1500	0	30
7876	ADAMS	CLERK	7788	23 MAY 87	1100		20
7900	JAMES	CLERK	7698	03 DEC 81	950		30
7902	FORD	ANALYST	7566	03 DEC 81	3000		20
7934	MILLER	CLERK	7782	23 JAN 82	1300		10

14 rows selected

Use a WHERE Clause to Filter Rows

GOOD (uses a WHERE clause to limit rows retrieved)

SELECT * FROM emp WHERE deptno IN (10, 20);

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17 DEC 80	800		20
7566	JONES	MANAGER	7839	02 APR 81	2975		20
7782	CLARK	MANAGER	7839	09 JUN 81	2450		10
7788	SCOTT	ANALYST	7566	19 APR 87	3000		20
7839	KING	PRESIDENT		17 NOV 81	5000		10
7876	ADAMS	CLERK	7788	23 MAY 87	1100		20
7902	FORD	ANALYST	7566	03 DEC 81	3000		20
7934	MILLER	CLERK	7782	23 JAN 82	1300		10

8 rows selected

Use Table Joins Rather than Multiple Queries

BAD (two separate queries when one would work)

```
SQL> select ename   from emp  where empno=7369;
```

ENAME
SMITH

```
SQL> select dname   from dept  where deptno=20;
```

DNAME
RESEARCH

GOOD (one query with join rather than two queries)

```
SQL>  select e.ename,d.dname   from emp e, dept d
      where e.deptno=d.deptno  and
      e.empno=7369;
```

ENAME	DNAME
SMITH	RESEARCH

Use Table Joins Rather than Multiple Queries

BAD (three separate queries when one CASE statement would work)

```
SQL> SELECT EMPNO,ENAME,SAL
     FROM EMP  WHERE SAL<=2000
```

EMPNO	ENAME	SAL
7369	SMITH	800
7499	ALLEN	1600
7521	WARD	1250
7654	MARTIN	1250
7844	TURNER	1500
7876	ADAMS	1100
7900	JAMES	950
7934	MILLER	1300

```
SQL> SELECT EMPNO,ENAME,SAL
     FROM EMP  WHERE SAL>2000
     AND SAL<=4000
```

EMPNO	ENAME	SAL
7566	JONES	2975
7698	BLAKE	2850
7782	CLARK	2450
7788	SCOTT	3000
7902	FORD	3000

```
SELECT EMPNO,ENAME,SAL   FROM EMP
WHERE SAL>4000
```

EMPNO	ENAME	SAL
7389	KING	5000

Use CASE Expressions Rather than Multiple Queries

BAD (three separate queries when one CASE statement would work)

```
SQL> SELECT COUNT(*)
       FROM EMP  WHERE SAL<=2000
```

COUNT(*)
8

```
SQL> SELECT COUNT(*)
       FROM EMP  WHERE SAL>2000
       AND SAL<=4000
```

COUNT(*)
5

```
SQL>SELECT COUNT(*) FROM EMP
      WHERE SAL>4000
```

COUNT(*)
1

GOOD (one query with a CASE expression rather than three queries)

```
SQL> SELECT
      COUNT(CASE WHEN SAL<=2000 THEN 1 ELSE null END) low,
     COUNT(CASE WHEN SAL>2000 AND SAL<4000 THEN 1 ELSE  null END) med,
     COUNT(CASE WHEN SAL>4000 THEN 1 ELSE null END) high
     FROM EMP
```

LOW	MED	HIGH
8	5	1

Use WHERE Rather than HAVING

BAD (Uses Having rather than where)

```
SQL>   SELECT JOB,MAX(SAL)   FROM   EMP
       GROUP BY JOB
        HAVING  JOB IN('SALESMAN','CLERK')
```

JOB	MAX(SAL)
CLERK	1300
SALESMAN	1600

GOOD (uses WHERE rather than HAVING)

```
SQL>  SELECT JOB,MAX(SAL)  FROM   EMP
       WHERE  JOB IN('SALESMAN','CLERK')
      GROUP BY JOB
```

JOB	MAX(SAL)
CLERK	1300
SALESMAN	1600

ACCESS PATH

- Access paths are ways in which data is retrieved from the database

Typical SQL operations

- *TABLE SCAN*
- *INDEX SCAN*
- *INDEX LOOKUP*

Typical SQL operations

TABLESCAN

```
mysql> explain format=tree select * from salgrade;
+                                                    +
| EXPLAIN                                            |
+                                                    +
|  > Table scan on salgrade  (cost=0.85 rows=6)      |
+                                                    +
```

contd..

- The command EXPLAIN FORMAT=TREE SELECT * FROM salgrade; in MySQL 8.0+ instructs the database to produce a human readable, hierarchical representation of how it intends to execute the query, rather than the traditional, hard to read tabular format.
- **Table scan (Access Method):** This means MySQL will read the table from beginning to end (full table scan) because there is no WHERE clause to filter rows or indexes to use.
- **cost=... (Query Cost):** This is an estimate of the resources (I/O, CPU) the query will use.
- **rows=... (Estimated Rows):** The optimizer predicts this many rows will be scanned
-

Typical SQL operations

INDEXSCAN

```
mysql> explain format=tree select deptno from dept;
+                                                    +
| EXPLAIN                                                                  |
+                                                    +
|  > Index scan on dept using PRIMARY  (cost=0.65 rows=4)  |
+                                                     +
```

Typical SQL operations

LOOKUP

```
mysql> explain format=tree select * from salgrade where
grade=1;
+                                                                    +
| EXPLAIN                                                            |
+                                                                    +
|  > Index lookup on salgrade using gra_idx (GRADE=1)  (cost=0.35 rows=1)   |
+                                                                    +
```

INDEXES

INDEXES CAN BE CREATED OR ALTERED

```
CREATE INDEX
        DAT_IDX ON
        EMP(hiredate)
```

```
ALTER  TABLE EMP
 ADD INDEX
        ENAME_IDX (ENAME)
```

INDEXES

Show index from test.emp

```
mysql> show index from test.emp;
+       +           +               +             +             +           +             +          +        +      +            +         +               +         +            +
| Table | Non_unique | Key_name       | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+       +           +               +             +             +           +             +          +        +      +            +         +               +         +            +
| emp   |          0 | PRIMARY        |            1 | EMPNO       | A         |          14 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| emp   |          1 | en_idx         |            1 | ENAME       | A         |          14 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
| emp   |          1 | comp1_idx      |            1 | DEPTNO      | A         |           3 |     NULL |   NULL | YES  | BTREE      |         |               | NO      | NULL       |
| emp   |          1 | comp1_idx      |            2 | ENAME       | A         |          14 |     NULL |   NULL | YES  | BTREE      |         |               | NO      | NULL       |
| emp   |          1 | comp_job_ename |            1 | ENAME       | A         |          14 |     NULL |   NULL | YES  | BTREE      |         |               | NO      | NULL       |
| emp   |          1 | comp_job_ename |            2 | JOB         | A         |          14 |     NULL |   NULL | YES  | BTREE      |         |               | NO      | NULL       |
| emp   |          1 | comp_job_ename |            3 | DEPTNO      | A         |          14 |     NULL |   NULL | YES  | BTREE      |         |               | NO      | NULL       |
+       +           +               +             +             +           +             +          +        +      +            +         +               +         +            +
7 rows in set (0.00 sec)

Mysql>
```

INDEXES

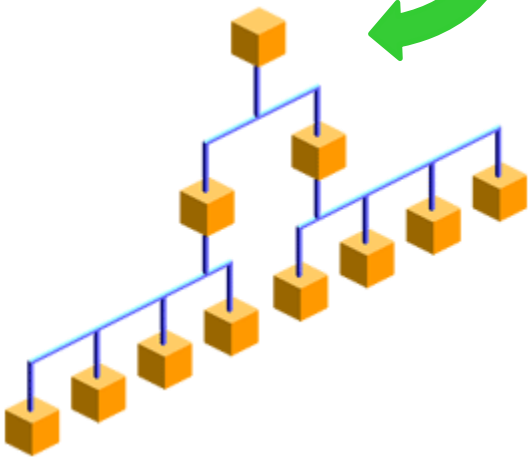
```
SELECT DISTINCT
TABLE_NAME,COLUMN_NAME,
   INDEX_NAME FROM
INFORMATION_SCHEMA.STATISTICS
WHERE TABLE_SCHEMA = 'test'
and table_name='emp';
```

TABLE_NAME	COLUMN_NAME	INDEX_NAME
emp	DEPTNO	fk_deptno
emp	EMPNO	PRIMARY

OPTIMIZER

Select * From emp
Where ename='SMITH'

OPTIMIZER



EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17 DEC 80	800		20
7499	ALLEN	SALESMAN	7698	20 FEB 81	1600	300	30
7521	WARD	SALESMAN	7698	20 FEB 81	1250	500	30
7566	JONES	MANAGER	7839	20 FEB 81	2975		20
7654	MARTIN	SALESMAN	7698	20 FEB 81	1250	1400	30
7698	BLAKE	MANAGER	7839	20 FEB 81	2850		30
7782	CLARK	MANAGER	7839	20 FEB 81	2450		10
7788	SCOTT	ANALYST	7566	20 FEB 81	3000		20
7839	KING	PRESIDENT		20 FEB 81	5000		10
7844	TURNER	SALESMAN	7698	20 FEB 81	1500		30
7876	ADAMS	CLERK	7788	20 FEB 81	1100		20
7900	JAMES	CLERK	7698	20 FEB 81	950		30
7902	FORD	ANALYST	7566	20 FEB 81	3000		20
7934	MILLER	CLERK	7682	20 FEB 81	1300		10

contd..

- The **MySQL Optimizer** is a built-in server component that acts as the "brain" of the database, determining the most efficient way to execute a given SQL query.
- The optimizer evaluates multiple potential execution paths and chooses the one it estimates will be the fastest and least resource.
- Optimizer is necessary because SQL is a declarative language—meaning you specify *what* data you want but not *how* to get it—

contd..

How the Optimizer Works

The optimizer operates as part of the query execution pipeline, following several critical steps:

1. **Preprocessing:** It verifies that tables and columns exist and resolves aliases and permissions.
2. **Query Transformation:** It may rewrite a query into an equivalent but more efficient form, such as converting subqueries into JOIN operations.
3. **Plan Generation:** It generates several possible execution plans by considering different indexing strategies, join orders, and algorithms.
4. **Cost Estimation:** Using a **Cost-Based Optimization (CBO)** model, it assigns a numerical cost to each plan based on estimated CPU and I/O usage.
5. **Plan Selection:** The plan with the lowest estimated cost is selected and passed to the execution engine.

contd..

Key Decisions Made by the Optimizer

- **Index Selection:** Decides whether to use an existing index or perform a full table scan based on data distribution.
- **Join Order:** Determines which table should be the "driving" table (processed first) to minimize intermediate result sizes.
- **Join Algorithms:** Chooses between methods like **Nested-Loop Join** (for small sets) or **Hash Join** (efficient for large sets in MySQL 8.0+).
- **Early Termination:** Identifies if a query can stop early, such as when using LIMIT or finding a single required value.

contd..

Controlling the Optimizer

While the optimizer is automated, developers and DBAs can influence its behavior:

- **EXPLAIN Statement:** Use the EXPLAIN command to see the execution plan the optimizer has chosen.
- **Optimizer Hints:** You can provide specific hints within a query (e.g., /*+ JOIN_ORDER(t1, t2) */) to force a particular behavior.
- **Update Statistics:** Because it relies on data snapshots, running ANALYZE TABLE ensures the optimizer has accurate, up-to-date information for its cost estimates.
- **Optimizer Switch:** Global or session-level settings (via optimizer_switch) can enable or disable specific features like **Index Merge** or **Semi-join**.

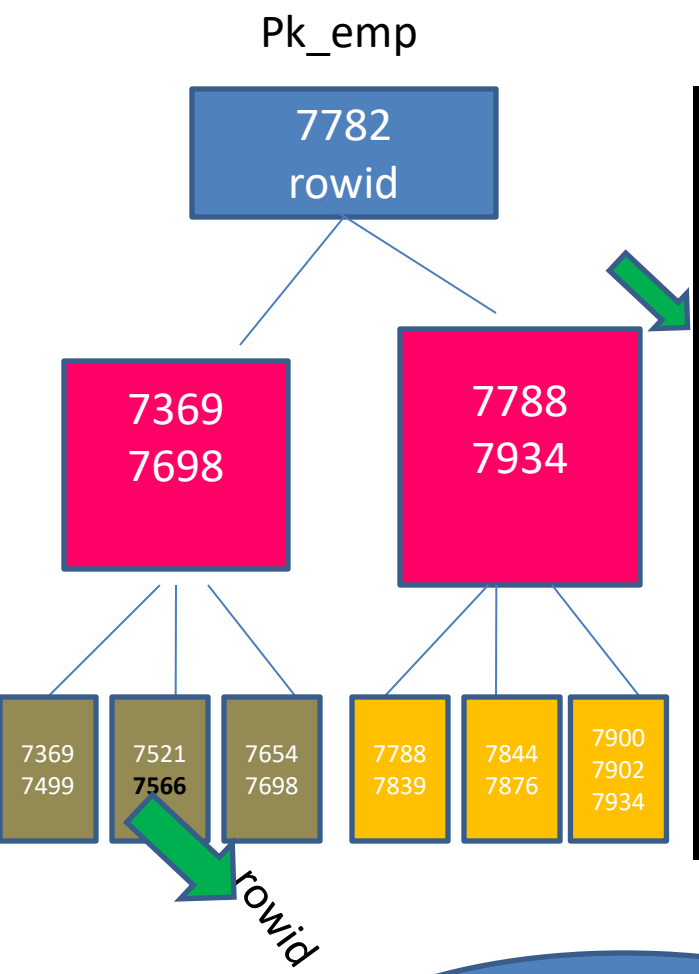
INDEXES

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17 DEC 80	800		20
7499	ALLEN	SALESMAN	7698	20 FEB 81	1600	300	30
7521	WARD	SALESMAN	7698	20 FEB 81	1250	500	30
7566	JONES	MANAGER	7839	20 FEB 81	2975		20
7654	MARTIN	SALESMAN	7698	20 FEB 81	1250	1400	30
7698	BLAKE	MANAGER	7839	20 FEB 81	2850		30
7782	CLARKE	MANAGER	7839	20 FEB 81	2450		10
7788	SCOTT	ANALYST	7566	20 FEB 81	3000		20
7839	KING	PRESIDENT		20 FEB 81	5000		10
7844	TURNER	SALESMAN	7698	20 FEB 81	1500		30
7876	ADAMS	CLERK	7788	20 FEB 81	1100		20
7900	JAMES	CLERK	7698	20 FEB 81	950		30
7902	FORD	ANALYST	7566	20 FEB 81	3000		20
7934	MILLER	CLERK	7682	20 FEB 81	1300		10

```
Select   ename
from     emp
where    empno=7566;
```

No Index on Empno column.
FTS will be done

INDEXES



EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17 DEC 80	800		20
7499	ALLEN	SALESMAN	7698	20 FEB 81	1600	300	30
7521	WARD	SALESMAN	7698	20 FEB 81	1250	500	30
7566	JONES	MANAGER	7839	20 FEB 81	2975		20
7654	MARTIN	SALESMAN	7698	20 FEB 81	1250	1400	30
7698	BLAKE	MANAGER	7839	20 FEB 81	2850		30
7782	CLARKE	MANAGER	7839	20 FEB 81	2450		10
7788	SCOTT	ANALYST	7566	20 FEB 81	3000		20
7839	KING	PRESIDENT		20 FEB 81	5000		10
7844	TURNER	SALESMAN	7698	20 FEB 81	1500		30
7876	ADAMS	CLERK	7788	20 FEB 81	1100		20
7900	JAMES	CLERK	7698	20 FEB 81	950		30
7902	FORD	ANALYST	7566	20 FEB 81	3000		20
7934	MILLER	CLERK	7682	20 FEB 81	1300		10

```
Select   ename
from     emp
where    empno=7566;
```

Index on Empno column.
Index scan will be done

INVISIBLE INDEXES

- **Invisible indexes are useful to temporarily hide an index from the query optimizer.**

ADVANTAGES

- **Invisible indexes can help to a great extent, in testing scenarios where the optimizer behaviour needs to be tested without the index. This does not need *dropping* and *recreating* the index.**

- **Indexes can be created for certain adhoc queries (queries that are fired very infrequently) and can be made invisible after the usage.**

INVISIBLE INDEXES

mysql> SELECT INDEX_NAME, IS_VISIBLE FROM INFORMATION_SCHEMA.STATISTICS WHERE TABLE_SCHEMA = 'TEST' AND TABLE_NAME = 'EMP';

```
+            +            +
| INDEX_NAME | IS_VISIBLE |
+            +            +
| en_idx     | YES        |
| PRIMARY    | YES        |
+            +            +
```

mysql> ALTER TABLE EMP ALTER INDEX EN_IDX INVISIBLE;

```
+            +            +
| INDEX_NAME | IS_VISIBLE |
+            +            +
| en_idx     | NO         |
| PRIMARY    | YES        |
+            +            +
```

INDEX MONITORING

- Usage of Indexes can be monitored
- Eliminating unused indexes cuts down the overhead for DML operations
- `SYS.SCHEMA_UNUSED_INDEXES`

INDEXES

FUNCTION BASED

- Select based on a indexed column will not use index if query contains function.

INDEXES

FUNCTION BASED

hir_IDX

X

INDEX IS NOT UTILIZED

SELECT * FROM EMP WHERE MONTHname(HIREDATE)= 'January';

INDEXES

FUNCTION BASED

ALTER TABLE EMP ADD INDEX((MONTHname(HIREDATE)));

create index fun_idx on emp((dayname(hiredate)));

FUNCTION

Drop index fun_idx on emp

PARTITION TABLES

*SELECT * FROM NEW_EMP WHERE JOB='CLERK'*

```
SELECT COUNT(*) FROM NEW_EMP
WHERE JOB='CLERK';
 COUNT(*)

   160
```

ROWS

15 | 48 | 49 | 48

CLERK | CLERK | CLERK | CLERK

445 | 446 | 447 | 448

PAGEID

PAGES contains even other jobs also

PARTITION TABLES

*SELECT * FROM JOB_PART WHERE JOB='CLERK'*

```
SELECT COUNT(*) FROM JOB_PART
WHERE JOB='CLERK';
 COUNT(*)

   160
```

ROWS

160

CLERK

468

PAGEID

PAGE COTNTAINS CLERK job only

PARTITION TABLES

- Technique to reduce the amount of I/O's by dividing the table into smaller subsets. To provide different storage characteristics for same logical data.
- Large table queries will be able to skip partitions that don't contain the data. Data is partitioned using partition key.
- After partitions are defined, DDL statements can access and manipulate individual partitions rather than entire tables or indexes
- Each partition of a table or index must have the same logical attributes, such as column names, data types, and constraints, but each partition can have separate physical attributes such as tablespaces

PARTITION TABLES

- Tables greater than 2GB

- Table containing historical data – current month's data is updatable and the other 11 months data is read only

PARTITION TABLES

TYPES
LIST
RANGE
HASH
COMPOSITE

PARTITION TABLES

Range partition

- Range of column values
- Partition key can be composed of 16 columns
- There cant be any gaps in the partition ranges
- All rows will have less than and not equal to the upper bound for the partition
- Attempts to insert record that do not fit into any defined partitions will give error.

PARTITION TABLES

Range partition example

```
CCREATE TABLE empHIREDATE
(empno  INT, HIREDATE date)
 PARTITION BY RANGE(year(HIREDATE))
 (
 PARTITION HIREDATE1981 VALUES LESS THAN (1981),
 PARTITION HIREDATE1982 VALUES LESS THAN (1982),
 PARTITION HIREDATEmax VALUES LESS THAN
maxvalue);
```

```
CREATE TABLE empsal
(empno  INT, sal INT)
 PARTITION BY RANGE(sal)
 (
 PARTITION SAL2000 VALUES LESS THAN
(2000),
 PARTITION SAL4000 VALUES LESS THAN
(4000),
 PARTITION SAL6000 VALUES LESS THAN
MAXVALUE);
```

PARTITION TABLES

List partition

- Set of specified values rather than range of values.
- Values which describe partition should be unique.
- User has explicit control over inserting rows
- Attempts to insert record that do not fit into any defined partitions will give error.

PARTITION TABLES

```
create table emp1 (empno number,ename varchar2(20),
job varchar2(10),mgr number, hiredate date,
sal number,comm number,deptno number)
partition by list( columns(job)
(partition emp_salesman values ('SALESMAN') TABLESPACE SK_USERS2,
PARTITION EMP_MANAGER VALUES('MANAGER') TABLESPACE SK_USERS,
PARTITION EMP_OTHER VALUES (DEFAULT) TABLESPACE USERS);
```

```
SQL> INSERT INTO EMP1 SELECT * FROM EMP;
13 rows created.
```

```
SQL> select partition_name, partition_ordinal_position, table_rows
from information_schema.partitions where table_name='EMP1'
```

TABLE_NAME	PARTITION_NAME	NUM_ROWS
EMP1	EMP_MANAGER	3
EMP1	EMP_OTHER	6
EMP1	EMP_SALESMAN	4

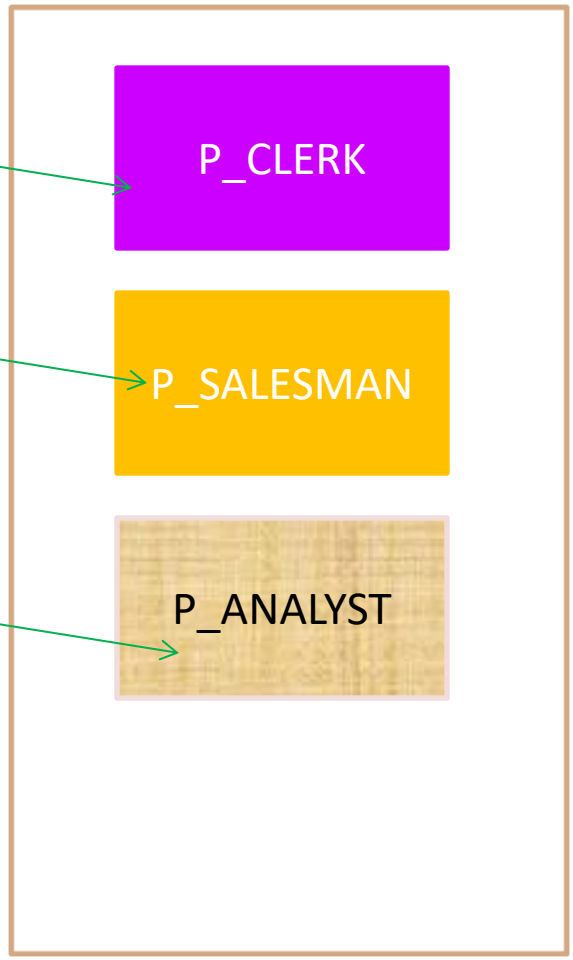
PARTITION TABLES

List partition

INSERT INTO EMP1 VALUES(1111,'test','CLERK')

INSERT INTO EMP1 VALUES(2222,'Best','SALESMAN')

INSERT INTO EMP1 VALUES(3333,'Rest','ANALYST')

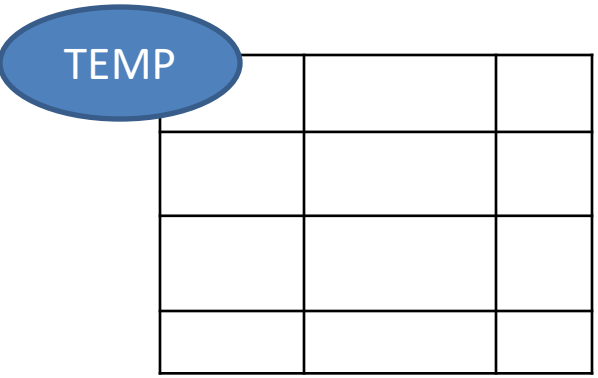


PARTITION TABLES

Hash partition

- Hashing algorithm is used to insert records
- Even number of records in each partitions
- Number of partitions has to be specified
- Applications retrieve the data from partitioned table using unique key

TEMPORARY TABLES




```
create temporary table temp_1 as select * from departments
SELECT *  FROM  temp_1;
```

department_id	department_name	manager_id	location_id
10	Administration	200	1700
20	Marketing	201	1800
30	Purchasing	114	1700
40	Human Resources	203	2400
50	Shipping	121	1500
60	IT	103	2300
70	Public Relations	204	2700
80	Sales	145	2500
90	Executive	100	1700
100	Finance	108	1700

Hold data just for

Session duration

```
mysql> show tables;
+                          +
| Tables_in_hr             |
+                          +
| 1_tab                    |
| account                  |
| agg_view                 |
```

Advantages Of Temp Tables

- Simplify Coding
- Simulate Cursors when Processing Data from Multiple Tables
- Improve Performance In a Situation Where Multi Table Joins are Needed
- Associate Rows from Multiple Queries in One Result Set (UNION)
- Eliminate Re Querying Data Needed for Joins
- Consolidate the Data for Decision Support Data Requirements
- A temporary table in MySQL will be dropped automatically when the user closes the session or terminates the connection manually.

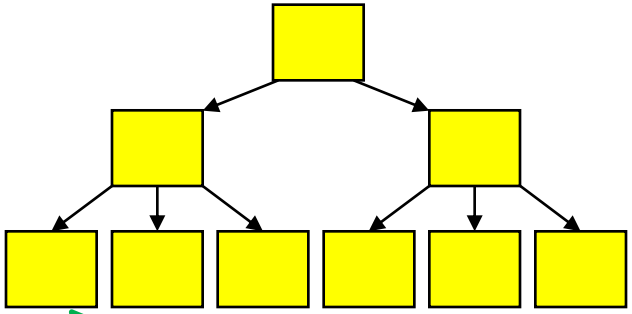
Using Profiler

MySQL query profiling is a useful technique when trying to analyze the overall performance of a database driven application.

```
SELECT @@profiling;
SET profiling = 1;
SELECT @@profiling;
SOME QUERIES
SHOW PROFILES;
SHOW PROFILE FOR QUERY1
SELECT STATE, FORMAT(DURATION, 6) AS DURATION FROM
INFORMATION_SCHEMA.PROFILING WHERE QUERY_ID = 2 ORDER BY SEQ;
```

INDEXES

EMPNO,DEPTNO



SELECT * FROM EMP
WHERE DEPTNO=10

X

SELECT * FROM EMP
WHERE EMPNO=7369

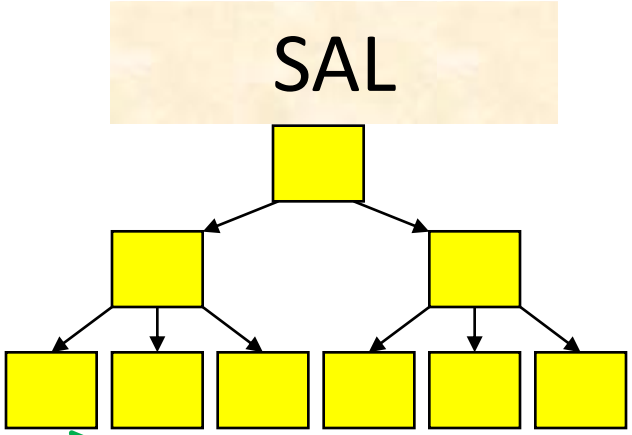
Y

LEADING COLUMNS WILL BE UTILIZING INDEX

INDEXES

SELECT * FROM EMP
WHERE SAL*1.5>3000

X



SELECT * FROM EMP
WHERE SAL>3000/1.5

Y

LHS SHOULD BE MATCHING INDEXED COLUMN

System variables

- **SHOW VARIABLES**: Tells the server to display its system configuration vars.
- **LIKE 'datadir'**: Filters the results to show only the variable named "datadir" (Data Directory).
- **Purpose:** The data directory contains subdirectories for each database you create, as well as InnoDB tablespace files, log files, and auto configuration files

innodb_page_size is a MySQL system variable that defines the size of the basic unit of storage (the "page") used by the InnoDB storage engine to store table rows, indexes, metadata, and undo logs. It determines the block size for both disk I/O and memory caching in the InnoDB Buffer Pool.

- **Default Size:** The default size is **16KB (16384 bytes)**, which is suitable for a wide range of workloads.

contd..

innodb_buffer_pool_size and innodb_buffer_page_size are two distinct but related MySQL/InnoDB configuration variables that determine how the database uses memory (RAM) and manages disk I/O, respectively.

1. innodb_buffer_pool_size

This is arguably the most crucial configuration parameter for MySQL performance.

- **Meaning:** It defines the total amount of RAM allocated to the InnoDB Storage Engine to cache data and indexes for tables.

2. innodb_buffer_page_size

- **Meaning:** This defines the unit size of data managed by InnoDB, representing how much data is read from or written to disk at once.
- **Default Size:** The default value is **16 KB**.

contd..

- **SELECT TABLESPACE_NAME, FILE_NAME, TABLE_NAME**: Specifies the data to retrieve from the INFORMATION_SCHEMA.FILES Table:
  - **TABLESPACE_NAME**: The name of the logical storage container (tablespace) where the table is stored (e.g., innodb_system or schema_name/table_name).
  - **FILE_NAME**: The actual path and name of the physical data file on the disk (e.g., a .ibd file).
  - **TABLE_NAME**: The name of the database table associated with that file.
- **FROM INFORMATION_SCHEMA.FILES**: Tells MySQL to look in the FILES table within the INFORMATION_SCHEMA database, which acts as a "data dictionary" containing system level metadata.

contd..

- In MySQL, specifically within the **InnoDB** storage engine, these parameters control how memory is allocated to speed up read and write operations. The innodb_log_buffer_size and innodb_log_file_size (often referred to
as log_buffer_size or log_file_size contextually) are crucial for tuning transaction performance and disk I/O, while the innodb_buffer_pool_size is the primary cache for data.

contd..

- select table_schema, sum((data_length+index_length)/1024/1024) AS MB from information_schema.tables group by 1;
- calculates the **total size of each database (schema)** in a MySQL or MariaDB instance and displays the result in megabytes (MB).
- **table_schema**: The name of the database.
- **sum((data_length + index_length) / 1024 / 1024) AS MB**:
  - **data_length**: The size of the actual data in bytes.
  - **index_length**: The size of all indexes associated with the table in bytes.
  - **/ 1024 / 1024**: Converts the byte values into megabytes.
  - **SUM(...)**: Aggregates the sizes of all tables within each schema.
- **FROM information_schema.tables**: Queries the INFORMATION_SCHEMA TABLES system table, which stores metadata about every table in the instance.
- **group by 1**: A shorthand for GROUP BY table_schema. It groups the summed sizes by the first column selected (the database name)

PRACTICE 1

Column	Datatype	Size
CUST_CODE	VARCHAR2	3
NAME	VARCHAR2	50
REGION	VARCHAR2	5

Column	Datatype	Size
ORD_ID	NUMBER	3
ORD_DATE	DATE	
CUST_CODE	VARCHAR2	3
DATE_OF_DELY	DATE	

Insert data into these tables through script 1.

1.Create appropriate indexes on name, region

2. Create index on ord_id

PRACTICE 1a

create and populate the NUMBERS table.(100000 rows)
b Query the table NUMBERS to find the number of distinct values in the two columns in the table.
c Create B tree indexes NUMB_OE_IDX and NUMB_NO_IDX on the ODD_EVEN and NO columns of the NUMBERS table, respectively, and check the total sizes of the indexes.
Hint: Check the total blocks allocated to the extents from DBA_SEGMENTS.
d Create bitmap indexes NUMB_OE_IDX and NUMB_NO_IDX on the ODD_EVEN and NO columns of the NUMBERS table, respectively, and check the total sizes of the indexes.
What can you conclude about the relationship between cardinality and sizes of the two types of indexes?

Column	Datatype	Size	CONSTRAINT
Numb	int		Primary key
type	varchar2	4	Check constraint ('odd','even')

PRACTICE 1b

Create index on hiredate column

a. **Select employee details who have joined in 1981**
b. **Using explain plan find out whether index is utilized or not**
c. **If not suggest remedy. Try to implement the remedy.**

PRACTICE 1c

Create index on job,deptno

a. **Find out employees working in deptno 30 as clerk**
b. **Using explain plan find out whether index is utilized or not**
c. **Find out employees working either in deptno 30 or as clerk.**
d. **Using explain plan find out whether index is utilized or not.**
e. **Use appropriate hint and check the explain plan**

PRACTICE 1d

1.FIND OUT EMPLOYEE NAME AND THEIR REPORTING OFFICERS NAME

A.USE EXPLAIN PLAN FIND OUT WHAT TYPE OF JOIN IT IS

B. USE HINTS FOR DIFFERENT TYPE OF JOINS

C. OBSERVE HOW COST IS REDUCED

2. CHANGE OPTIMIZER MODE

CHOOSE

FIRST_ROWS

3. TRY TO CREATE INDEX ON ENAME

A. USE EXPLAIN PLAN AND FIND OUT THE COST

PRACTICE 1e

1.FIND OUT EMPLOYEES WHO ARE TAKING SAME SALARY AS THAT OF MR.MARTIN

A.USE EXPLAIN PLAN FIND OUT COST

B. TUNE THE QUERY TO REDUCE THE COST

2.FIND OUT EMPLOYEES WHO ARE WORKING IN THE SAME DEPARTMENT AS MR.BLAKE

A.USE EXPLAIN PLAN FIND OUT COST

B. TUNE THE QUERY TO REDUCE THE COST

PRACTICE 1f

1. Create a table called STUDENT1

Column name	Datatype	Size	CONSTRAINT
Student_id	NUMBER	4	PRIMARY KEY
Student_name	Varchar2	15	
Result	Char	1	Check 'P','F'

a. Copy the rows from emp1 table for student_id,student_name

b. Update the result with 'P' for name containing 'S'. For others enter 'F'

c. Display number of students along with each type of result

d. Using explain plan find out the operation

e. Suggest recommendations and implement it to reduce the cost

PRACTICE 1g

1.Find out the job availabLe in deptno 10.

a.Find out the cost using explain plan

b.Rewrite the query using set operator

c.Find out the cost using explain plan

d. Create index and reduce the cost

PRACTICE 1h

1.Find out the employee details who have joined between ‘01 jan 81’ and ‘31 dec 81’

a.Find out the cost using explain plan

b. Create index and reduce the cost

PRACTICE 1i

1.Find out the employee details who have empno greater than 7839

a. Using explain plan find out the steps involved in processing the query

PRACTICE 1j

1.Find out the employee who are working in SALES department.

a. Using subquery
b. Using Joins
c. Find out the cost for both
d. Tune the query

PRACTICE 1k

1.Display the ename,salary,salary*1.5 from emp table.

a. Name salary*1.5 as newsal
b. Find out where newsal is greater than 3000
c. Using explain plan find out the operations involved
d. create index on sal column. Find out whether index is utilized or not
e. Suggest remedies

PRACTICE 1l

1.Print list of employees displaying "Good salary" if sal > 3000, "Average salary" if sal=3000 and "Poor salary" if sal < 3000.

a. Using explain plan find out the operations involved

b. Display the above data only for deptno 30

c. Using explain plan find out the operations involved

d. Tune the query

PRACTICE 1m

Create table new_emp as a replica of emp

1. Copy rows from new_emp table excluding empno 7839(about 200000 rows)
2. Find out number of employees in each job. Using explain plan find out the operations, cost involved.
3. Tune the query by creating index on job column.
4. Drop the index and create bitmap index. Using explain plan find out the operations/cost involved

PRACTICE 1n

Find out number of employees working in SALES AND RESEARCH department (USE new_emp and dept table)

1. Using explain plan find out the operations, cost involved.
2. Tune the query by creating appropriate indexes .
3. Drop the indexes and create bitmap index. Using explain plan find out the operations/cost involved

PRACTICE 1o

Find out number of employees working in chicago as clerk(USE new_emp and dept table)

1. Using explain plan find out the operations, cost involved.
2. Tune the query by creating appropriate indexes .
3. Find out whether index on job is utilized or not.
4. Use hint to force optimizer to use index on job and find out cost.

PRACTICE 1p

Open session 1 and 2 as scott. Run the following queries.

1. **Select * from emp.**
2. **Select sal from emp**
3. **Open session 1 , run**

   select * from emp where sal=3000

   Open session 2 ,run

   select * from emp where sal=1250;

   a. Check using suitable data dictionary whether the soft parsing is done or hard parsing done in all the 3 cases.

   b. If hard parsing in case 3 can we implement soft parsing.

PRACTICE 1Q

FIND OUT EMPLOYEES WHO ARE WORKING AS CLERK IN DEPTNO 20

CHECK THE PLAN
MAKE DEPTNO INDEX VISIBLE
AND CHECK THE PLAN
MAKE AGAIN INVISIBLE (DEPTNO INDEX)
MAKE JOB INDEX VISIBLE
AND CHECK THE PLAN.
MAKE BOTH VISIBLE
AND CHECK THE PLAN
MAKE COMPOSITE INDEX INVISIBLE
AND CHECK THE PLAN
MAKE ALL INDEXES INVISIBLE AND CHECK THE PLAN

JOIN METHODS

Access Methods Nested Loops

```
Select  eNAME
from   Emp e, Dept d
Where  d.deptno = e.deptno
and
Dname in ('ACCOUNTING' ,'RESEARCH');
```

Dept

DEPTNO	DNAME
10	ACCOUNTING
20	RESEARCH
30	SALES
40	OPERATIONS

Emp

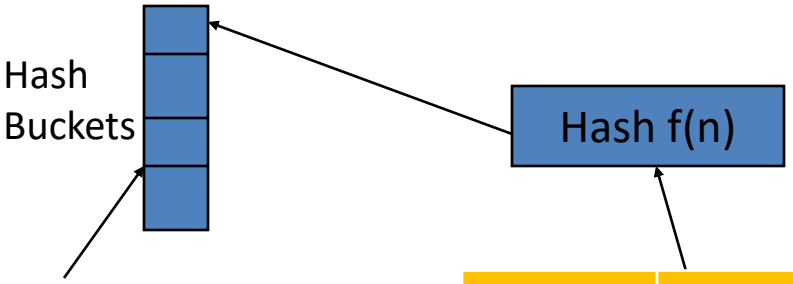
ENAME	DEPTNO
CLARK	10
KING	10
MILLER	10
SMITH	20
JONES	20
SCOTT	20
ADAMS	20
FORD	20

```
FORALL records in Dept
where dname in ('ACCOUNTING' ,'RESEARCH');
LOOP:
    FORALL records in EMP
    LOOP:
        where emp.deptno = dept.deptno
        Return record;
    END LOOP;
END LOOP;
```

JOIN METHODS

Access Methods Hash Join

```
Select  eNAME
from   Emp e, Dept d
Where  d.deptno = e.deptno
and
Dname in ('ACCOUNTING' ,'RESEARCH');
```



DEPTNO	DNAME
10	ACCOUNTING
20	RESEARCH
30	SALES
40	OPERATIONS

ENAME	DEPTNO
CLARK	10
KING	10
MILLER	10
SMITH	20
JONES	20
SCOTT	20
ADAMS	20
FORD	20

1. Take smaller of two tables
2. Create Hash Buckets in Shared Pool for join column of small table
3. Full Scan of larger table, applying Hash Function on join column
4. If Hash Function gives a hit in Hash Buckets, return row
   Else ignore row and continue scan